

Excelerate Data Analysis — Week 2

Data Integration and Cleaning Report

1. Data Integration Process

The Learner Dataset and Opportunities Dataset were combined into a single joined_table, selecting the fields listed below from each source table.

1.1 Fields included in the joined table

#	Learner Dataset	Opportunities Dataset
1	sk	opportunity_id
2	learner_fee	Badge
3	status	CareerAddOn
4	amount to be paid	opportunity_category
5	transaction id	code
6	payment status	currency_type
7	apply date	DropoutTransaction
8	from where did you hear about us	duration
9	accept reject reason	duration_type
10	learner category	Eligibility
11	reward awarded date	opportunity_fee
12	completion date	is archived
13	withdraw reason	is auto approve
14	application id	last date to apply
15	please identify your student status	location
16	why do you want to apply for this internship1	microscholarship
17	from which medium did you hear about the internship	name
18	why do you want to apply for this internship	Panellist
19	withdraw date	Reward
20	learner created at	role
21	work item sk	role responsibility
22	how_did_you_hear_about_this_opportunity	
23	dropped_out_date	
24	appear_in_wishlist	

1.2 Join logic used

Both datasets use a prefixed key format (e.g. Learner#... and Opportunity#...). A direct equality join on the raw sk and opportunity_id values does not work because of the differing prefixes, so the key was normalized first: each value was split on the "#" character and only the second part (the unique ID suffix) was compared. This is the field that actually links a learner record to the opportunity it belongs to.

```
-- checking which columns intersected
SELECT column_name
FROM information_schema.columns
WHERE table_name = 'learner_data'
INTERSECT
SELECT column_name
FROM information_schema.columns
WHERE table_name = 'opportunities_data';
```

Figure 1: Checking which column names are shared between the two tables before deciding on a join strategy.

The two tables were joined using an inner join, matching `SPLIT_PART(sk, '#', 2)` from the Learner Dataset to `SPLIT_PART(opportunity_id, '#', 2)` from the Opportunities Dataset, producing a new column, `clean_id`.

```
CREATE TABLE joined_table AS
SELECT
  SPLIT_PART(t1.sk, '#', 2) AS clean_id,
  t1.sk, t1.fee AS learner_fee, t1.status, t1.amount_to_be_paid, t1.transaction_id, t1.payment_status,
  t1.apply_date, t1.from_where_did_you_hear_about_us, t1.accept_reject_reason, t1.category AS learner_category,
  t1.reward_awarded_date, t1.completion_date, t1.withdraw_reason, t1.application_id,
  t1.please_identify_your_student_status, t1.why_do_you_want_to_apply_for_this_internship1,
  t1.from_which_medium_did_you_hear_about_the_internship, t1.why_do_you_want_to_apply_for_this_internship,
  t1.withdraw_date, t1.created_at AS learner_created_at, t1.work_item_sk,
  t1.how_did_you_hear_about_this_opportunity, t1.dropped_out_date, t1.appear_in_wishlist, t2.opportunity_id,
  t2."Badge", t2."CareerAddOn", t2.category AS opportunity_category, t2.code, t2.currency_type,
  t2."DropoutTransaction", t2.duration, t2.duration_type, t2."Eligibility", t2.fee AS opportunity_fee,
  t2.is_archived, t2.is_auto_approve, t2.last_date_to_apply, t2.location, t2.microscholarship, t2.name,
  t2."Panellist", t2."Reward", t2.role, t2.role_responsibility
FROM learner_data AS t1
INNER JOIN opportunities_data AS t2
  ON SPLIT_PART(t2.opportunity_id, '#', 2)
  = SPLIT_PART(t1.sk, '#', 2);
```

Figure 2: Join query — `CREATE TABLE joined_table` selecting the mapped fields from both sources, joined on the normalized ID suffix.

1.3 Data check after joining

Initial record count before joining:

- Learner data: 15,029 records, with `sk` having 184 null and 46 empty-string values.
- Opportunity data: 5,733 records, with `opportunity_id` having 3 empty-string values.

Record count after joining:

- 14,937 records after the inner join, with `clean_id` having 138 empty strings — a result of the 46 and 3 empty-string keys multiplying together during the join. Null key records were discarded by the inner join.
- 14,799 records remained after removing the rows with an empty `clean_id`.

```
-- new table has 46 columns and 14799 rows
SELECT *
FROM information_schema.columns
WHERE table_schema = 'public'
AND table_name = 'joined_table';
```

Figure 3: Confirming the new `joined_table` structure via `information_schema` after the join.

2. Data Cleaning Activities

With the datasets integrated into `joined_table`, each field was reviewed and cleaned according to its content and intended use.

2.1 Data type conversions

The following fields were converted from text to numeric, with null fees assumed to be zero:

- `learner_fee`
- `amount_to_be_paid`
- `duration`
- `microscholarship`

Additionally, the following fields were converted to timezone-aware timestamps:

- `apply_date`
- `reward_awarded_date`
- `withdraw_date`
- `learner_created_at`
- `dropped_out_date`
- `last_date_to_apply`

2.2 Categorical and status fields

- `status` was mapped to its corresponding label using a status-mapping reference table. Codes 103 and 110 were assumed to be missing a trailing zero and were corrected to 1030 and 1100 respectively. Null values (5,749 records) were left as-is rather than guessed at.
- `transaction_id`: placeholder strings and random gibberish values were converted to null.
- `work_item_sk`: invalid keys were converted to null.
- `appear_in_wishlist`: since no genuinely valid values were present, null records were assumed to mean "no" and populated records were assumed to mean "yes".
- `Badge` (renamed `badge` for naming consistency): populated with JSON strings, so a populated value was assumed to mean a badge was present, and null was assumed to mean no badge.
- `Eligibility` (renamed `eligibility`): populated records were assumed eligible; null records were assumed not eligible.
- `is_auto_approve`: null values were assumed to be false.
- `location`: standardized, and placeholder/test values were converted to null.
- `Reward` (renamed `reward`): populated fields were assumed to indicate a reward was offered and converted to 'yes'; null fields were converted to 'no'.
- `role`: only valid records were kept; invalid records were removed.

2.3 Fields left unchanged

The following fields were reviewed and did not require any changes:

- `payment_status`
- `from_where_did_you_hear_about_us`

- learner_category
- please_identify_your_student_status
- from_which_medium_did_you_hear_about_the_internship
- opportunity_category
- code
- currency_type
- role_responsibility — deliberately left unedited/unstandardized to avoid losing potentially useful free-text detail.

2.4 Noise filtering and dropped fields

Most of the noise (irrelevant or low-quality free text) was filtered out of the following fields:

- accept_reject_reason
- withdraw_reason
- why_do_you_want_to_apply_for_this_internship1
- why_do_you_want_to_apply_for_this_internship
- how_did_you_hear_about_this_opportunity

The following fields were dropped entirely, with the reasoning noted for each:

- application_id — contained repetitive dummy numbers; once filtered, only unusable gibberish remained.
- CareerAddOn — contained only JSON strings with no usable content.
- name and Panellist — determined not to be required for analysis.

Minor renames for consistency: DropoutTransaction → dropout_transaction; duration_type was edited for internal consistency.

2.5 Result of cleaning

The record count remained at 14,799 throughout the cleaning process (no additional rows were dropped during field-level cleaning). The number of fields was reduced from the original selection down to the final cleaned set after dropping application_id, CareerAddOn, name, and Panellist.

column_name name	ordinal_position integer	column_default character varying	is_nullable character varying (3)	data_type character varying	cf in
clean_id	1	[null]	YES	text	
sk	2	[null]	YES	text	
learner_fee	3	[null]	YES	numeric	
status	4	[null]	YES	text	
amount_to_be_paid	5	[null]	YES	numeric	
transaction_id	6	[null]	YES	text	
payment_status	7	[null]	YES	text	
apply_date	8	[null]	YES	timestamp with time zone	
from_where_did_you_hear_about_us	9	[null]	YES	text	
accept_reject_reason	10	[null]	YES	text	
learner_category	11	[null]	YES	text	
reward_awarded_date	12	[null]	YES	timestamp with time zone	
completion_date	13	[null]	YES	timestamp with time zone	
withdraw_reason	14	[null]	YES	text	
please_identify_your_student_status	16	[null]	YES	text	

Figure 4: Snapshot of the cleaned joined_table column list, showing the corrected data types (numeric, timestamp with time zone, text).

3. Validation & Comparison

Comparing the dataset before and after the integration and cleaning process performed above:

Aspect	Before	After
Learner records	15,029 (184 null sk, 46 empty sk)	N/A — merged into joined_table
Opportunity records	5,733 (3 empty opportunity_id)	N/A — merged into joined_table
Joined records	N/A	14,937 (raw inner join)
Records after removing empty clean_id	N/A	14,799
Records after full field cleaning	N/A	14,799 (unchanged)
Key fields with unreliable/unusable text	transaction_id, application_id, work_item_sk, CareerAddOn, name, Panellist — unfiltered	Cleaned, standardized, or dropped where unusable
Numeric fields stored as text	learner_fee, amount_to_be_paid, duration, microscholarship, opportunity_fee	Converted to numeric type

Improvements made: the two source tables, which had no directly compatible key format, are now integrated into a single queryable joined_table using a normalized ID match. Numeric and date fields that were previously stored as inconsistent text are now properly typed, enabling direct aggregation and time-based analysis. Noisy free-text and low-value fields were either cleaned or removed, reducing clutter without discarding fields (like role_responsibility) that may still hold analytical value.

Validation checks performed: row counts were confirmed at each stage of the join (before join, after join, after removing empty keys); the final column list and data types were confirmed via information_schema.columns (Figure 4) to ensure the type conversions applied correctly.

4. Updated Analysis

With joined_table cleaned and properly typed, it is ready to support direct SQL analysis and dashboard/reporting use — for example, aggregating learner_fee or amount_to_be_paid by opportunity_category, or trending apply_date and completion_date over time now that both are true timestamp fields rather than text.

The specific queries used to build and validate the cleaned joined_table (including the status-mapping logic and the field-by-field cleaning steps summarized in Section 2) are documented in full at the link below:

[Query documentation — Google Doc](#)

Note: the record and field counts documented in this report (15,029 / 5,733 source records; 14,799 joined and cleaned records) reflect the dataset state at the time this integration was performed, and may differ slightly from row counts reported in the Week 1 EDA if additional cleaning was applied to either source table between the two stages.

5. Conclusion

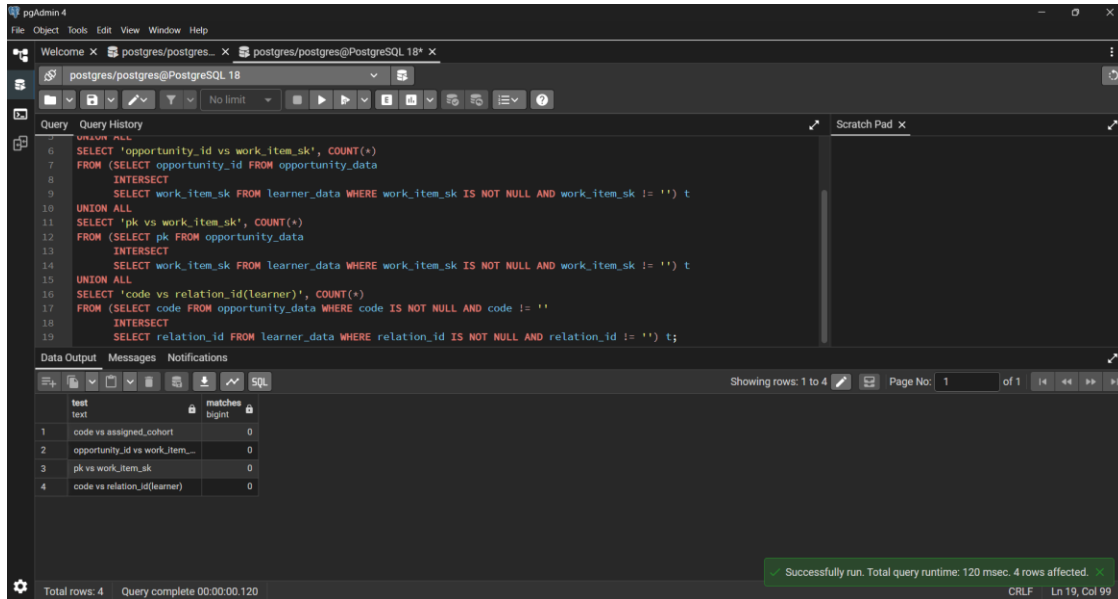
This report has documented the full Week 2 data integration and cleaning process for the Excelerate Opportunity Data and Learner Data. An initial investigation confirmed that no direct, reliably matching identifier exists between the two source exports; normalizing the sk and opportunity_id keys with SPLIT_PART on the shared "#" delimiter resolved this and produced a working inner join, taking the dataset from 15,029 learner and 5,733 opportunity records down to 14,799 cleanly joined rows. Field-level cleaning then addressed inconsistent data types, undocumented status codes, unusable free text, and redundant columns, without any further loss of records.

The resulting joined_table is properly typed, de-duplicated of unusable fields, and validated at every stage of the process, satisfying the goal of Week 2: moving from two independently messy source tables to a single, analysis-ready dataset. This directly sets up the work for Week 3, where the cleaned joined_table can be used for deeper trend analysis and dashboard development, alongside resolving the remaining open item of confirming completion_date behaves correctly in the new joined structure.

Appendix: Join-Key Investigation Trail

Before arriving at the SPLIT_PART-based join documented in Section 1.2, an independent investigation was carried out directly against the PostgreSQL tables to identify a working join key. That process is documented here for completeness, since it explains why a normalized-key approach was necessary rather than a direct equality join.

Several named candidate columns (pk, opportunity_id, code, cohort/relation_id, work_item_sk, application_id) were tested pairwise using INTERSECT queries on their raw values. All returned zero matches:



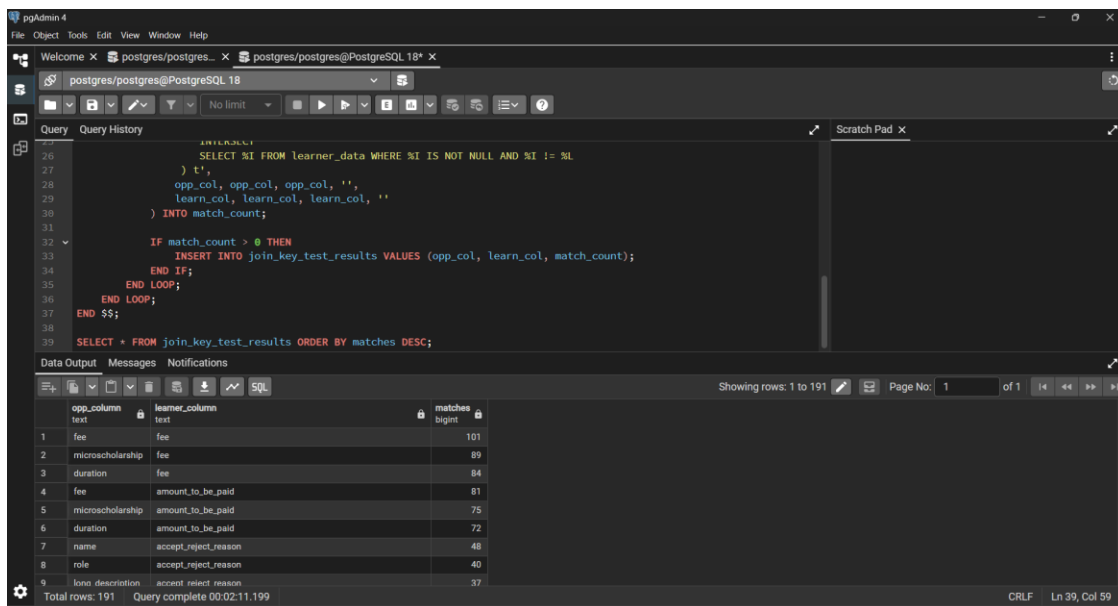
```
6 SELECT 'opportunity_id vs work_item_sk', COUNT(*)
7 FROM (SELECT opportunity_id FROM opportunity_data
8 INTERSECT
9 SELECT work_item_sk FROM learner_data WHERE work_item_sk IS NOT NULL AND work_item_sk != '') t
10 UNION ALL
11 SELECT 'pk vs work_item_sk', COUNT(*)
12 FROM (SELECT pk FROM opportunity_data
13 INTERSECT
14 SELECT work_item_sk FROM learner_data WHERE work_item_sk IS NOT NULL AND work_item_sk != '') t
15 UNION ALL
16 SELECT 'code vs relation_id(learner)', COUNT(*)
17 FROM (SELECT code FROM opportunity_data WHERE code IS NOT NULL AND code != ''
18 INTERSECT
19 SELECT relation_id FROM learner_data WHERE relation_id IS NOT NULL AND relation_id != '') t;
```

test	matches
code vs assigned_cohort	0
opportunity_id vs work_item...	0
pk vs work_item_sk	0
code vs relation_id(learner)	0

Successfully run. Total query runtime: 120 msec. 4 rows affected.

Figure A1: Four initial ID-candidate pairs tested by raw value equality — all returned zero matches.

Every text column in Opportunity Data was then cross-checked against every text column in Learner Data using a PL/pgSQL loop, to rule out any relationship not captured by column naming alone:



```
26 INTERSECT
27 SELECT %i FROM learner_data WHERE %i IS NOT NULL AND %i != %l
28 ) t',
29 app_col, opp_col, opp_col, '',
30 learn_col, learn_col, learn_col, ''
31 ) INTO match_count;
32 IF match_count > 0 THEN
33 INSERT INTO join_key_test_results VALUES (opp_col, learn_col, match_count);
34 END IF;
35 END LOOP;
36 END LOOP;
37 END $$;
```

opp_column	learner_column	matches
fee	fee	101
microscholarship	fee	89
duration	fee	84
fee	amount_to_be_paid	81
microscholarship	amount_to_be_paid	75
duration	amount_to_be_paid	72
name	accept_reject_reason	48
role	accept_reject_reason	40
long description	accept_reject_reason	37

Total rows: 191 Query complete 00:02:11.199

Figure A2: Full cross-column comparison across both tables — top results were coincidental matches on short categorical values, not real identifiers.

Filtering that result down to identifier-like columns only still showed just a handful of matches — including `opportunity_id` vs. `sk` at 2 matches, which in hindsight was the real relationship, just undercounted because the raw values still carried their differing `Learner#/Opportunity#` prefixes:

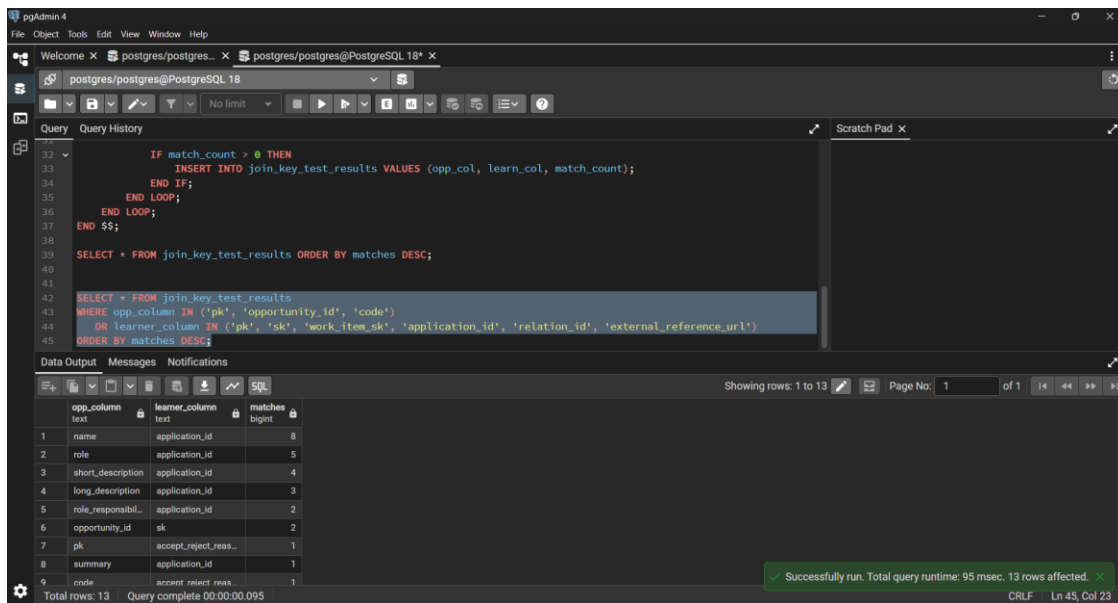


Figure A3: ID-like column pairs only. `opportunity_id` vs. `sk` appears here with a low raw match count — the prefix mismatch was hiding the real relationship later confirmed via `SPLIT_PART`.

As a parallel approach while the row-level key was still unresolved, a category-level aggregation join was also built and validated directly in PostgreSQL, combining opportunity and learner counts per category:

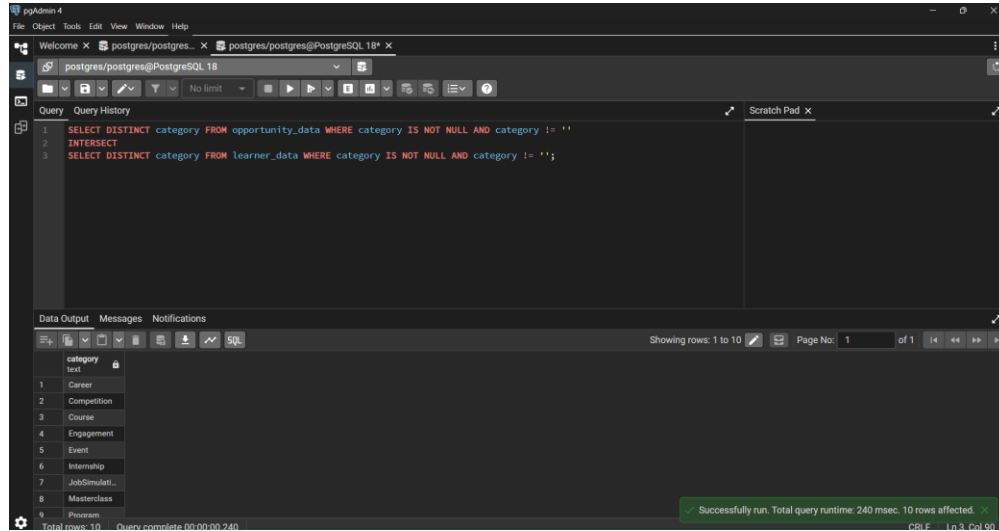


Figure A4: Confirming 10 overlapping category values between the two tables, used as the basis for a category-level fallback join.

The screenshot shows the pgAdmin 4 interface with a query window containing the following SQL:

```
SELECT * FROM integrated_category_summary;
```

The Data Output pane displays the following table:

category	total_opportunities	opportunities_with_reward	total_applications	completed_applications	applications_per_opportunity
text	bigint	bigint	bigint	bigint	numeric
1 Internship	1591	1591	1593	0	1.00
2 Career	1088	1087	1088	0	1.00
3 Competition	734	734	741	0	1.01
4 Course	632	632	638	0	1.01
5 Event	622	622	628	0	1.01
6 Engagement	459	459	459	0	1.00
7 Masterclass	400	400	400	0	1.00
8 JobSimulation	198	198	198	0	1.00
9 Xolomni	2	2	2	0	1.00

The status bar indicates: "Successfully run. Total query runtime: 127 msec. 16 rows affected." and "CRLF Ln 1, Col 43".

Figure A5: Category-level integrated summary produced from the fallback approach — superseded by the row-level `SPLIT_PART` join once it was identified.

This category-level view surfaced a data quality issue worth carrying forward: a `completed_applications` aggregation returned zero for every category, despite `completion_date` being populated on 1,132 learner records:

The screenshot shows the pgAdmin 4 interface with a query window containing the following SQL:

```
SELECT COUNT(*) FROM learner_data
WHERE completion_date IS NOT NULL AND completion_date != '';
```

The Data Output pane displays the following table:

count
bigint
1 1132

The status bar indicates: "Successfully run. Total query runtime: 109 msec. 1 rows affected." and "CRLF Ln 2, Col 61".

Figure A6: Direct count confirming 1,132 learner `data` rows have a non-blank `completion_date`, contradicting the 0 values in the category-level view — worth re-checking against the new `joined_table`'s `completion_date` column.